

System Architecture Documentation Pipeline

Step 0) Create the architecture documentation workspace first

Goal

Set up the place where all architecture work will live before you start drawing or writing.

Deliverables

- Documentation root
- ADR folder
- Diagram source folder
- Database design folder
- Event catalog folder
- API contract folder

Tools

- **Git repository** (best default)
- **Markdown**
- **Mermaid / PlantUML / Structurizr DSL** for diagrams
- Optional rendered exports: PNG/SVG/PDF
- Optional publishing layer: Confluence / Notion / Azure DevOps Wiki / GitHub Pages

Why this comes first

Because architecture documentation should be **versioned, reviewable, diffable, and tied to code changes**. ADR guidance explicitly recommends storing ADRs in Git and commonly places them under `adr/` or `docs/adr/`. [\[github.com\]](https://github.com), [\[adr.github.io\]](https://adr.github.io)

My recommendation

Use **docs-as-code in the same repo as the backend codebase** as the **source of truth**. Then optionally publish a rendered copy to Confluence/Notion for non-engineering stakeholders. This keeps design and implementation from drifting.

Step 1) Extract requirements and architecturally significant requirements (ASRs)

Goal

Turn the project brief into a precise list of:

- Functional requirements
- Non-functional requirements (NFRs)
- Constraints
- Assumptions
- Risks
- Open questions

Deliverables

- 01-requirements-and-goals.md
- Functional scope list
- NFR list
- Constraints section
- Assumptions log
- Glossary

Tools

- Jira / Azure DevOps / Linear / Notion
- Markdown
- Interview notes / workshop notes

What to document

At minimum:

- Core use cases
- Expected user roles
- Throughput / latency expectations
- Scalability expectations
- Availability expectations
- Security/compliance expectations
- Integration requirements (Stripe, Elasticsearch, Redis/BullMQ, auth)
- Operational constraints (cloud, cost, deployment model, team size)

Why this is industry-standard

ISO 42010 centers architecture descriptions around **stakeholders and concerns**, and arc42 starts with **Introduction and Goals, Constraints**, and stakeholder-driven quality goals.

[\[iso.org\]](https://iso.org), [\[arc42.org\]](https://arc42.org)

Practical tip

Write NFRs as **measurable scenarios**, not vague goals:

- “95th percentile read API latency < 200ms”
 - “Webhook event processing is idempotent”
 - “System tolerates duplicate event delivery”
 - “Search results visible within 5 seconds of data mutation”
-

Step 2) Identify stakeholders, actors, and system boundaries

Goal

Define who interacts with the system and where the system starts/stops.

Deliverables

- Stakeholder list
- Actor list
- External systems list
- Context notes

Tools

- Miro / diagrams.net / Excalidraw
- Markdown

What to capture

- End users
- Admin/support users
- Finance or operations roles
- External systems (Stripe, email/SMS provider, Elasticsearch cluster, Redis, internal admin systems)
- Trust boundaries (internet, internal network, private VPC, third-party boundary)

Why

This is the basis for the **C4 System Context diagram**, which exists specifically to answer: What is the scope of the system, who uses it, and what external systems does it integrate with? [\[c4model.com\]](https://c4model.com)

Step 3) Define the top-level architecture strategy and write ADRs for major choices

Goal

Capture the “why” behind the big decisions before detailed design begins.

Deliverables

Initial ADRs such as:

- 0001-use-modular-monolith.md
- 0002-use-postgres-as-primary-rdbms.md
- 0003-use-redis-bullmq-for-background-jobs.md
- 0004-use-elasticsearch-for-search-read-models.md
- 0005-use-domain-events-and-outbox-pattern.md
- 0006-use-jwt-for-authentication.md
- 0007-use-rbac-plus-resource-level-policies.md
- 0008-use-stripe-with-webhook-driven-reconciliation.md

Tools

- Markdown
- ADR template (MADR, Nygard, or similar)
- Git pull requests

What each ADR should contain

- Context
- Decision
- Alternatives considered
- Consequences
- Risks / trade-offs
- Status (Proposed / Accepted / Superseded)

Why

An ADR is explicitly meant to capture a **single important architectural decision together with its rationale and consequences**, and the collection of ADRs becomes the project’s decision log. [\[adr.github.io\]](https://adr.github.io/), [\[github.com\]](https://github.com)

Important note

Do this early. If you skip ADRs, architecture becomes tribal knowledge.

Step 4) Draw the C4 System Context diagram

Goal

Show the system from the outside.

Deliverables

- system-context.mmd or system-context.dsl
- Exported PNG/SVG if needed

Tools

- Structurizr DSL (excellent if you want architecture-as-code)
- Mermaid / PlantUML / draw.io

What it should show

- Your backend system as one central software system
- Actors (customer, admin, operations, support, finance)
- External dependencies:
 - Stripe
 - Email/SMS provider
 - Elasticsearch
 - Redis
 - Identity provider (if any)
 - Internal dashboards / back office systems

Why

In C4, this is the first diagram and it clarifies scope and integrations for both technical and non-technical stakeholders. [\[c4model.com\]](https://c4model.com)

Step 5) Design the modular monolith decomposition (modules / bounded contexts)

Goal

Break the monolith into explicit modules with clear ownership and boundaries.

Deliverables

- module-map.md
- component-diagram (C4 Component-level for the backend)
- Dependency rules
- Ownership matrix

Tools

- diagrams.net / Structurizr DSL / Mermaid
- Markdown
- Event storming session board (Miro) if the domain is complex

What to define

You want modules like:

- Identity / Authentication
- Authorization / Access Control
- Users / Organizations / Tenancy
- Catalog / Domain entities
- Orders / Booking / Workflow
- Payments / Billing
- Notifications
- Search indexing
- Background jobs
- Audit / Compliance
- Webhook ingestion
- Reporting / Read models

What “good” looks like in a modular monolith

Each module should have:

- A clear responsibility
- Its own application services
- Its own domain model
- Controlled dependencies
- Its own data ownership rules
- Internal events rather than random cross-module method calls

Practical rule

Prefer:

- **Synchronous calls** only for immediate consistency where required
 - **Domain events + background processing** for side effects, search indexing, notification fan-out, audit logging, and payment post-processing
-

Step 6) Create the C4 Container diagram

Goal

Show the major deployable/runtime building blocks.

Deliverables

- container-diagram.mmd
- container-responsibilities.md

Tools

- C4 tooling (Structurizr, Mermaid, PlantUML, draw.io)

What to include for your stack

Although it is a modular monolith, the Container diagram should still show the major runtime containers/data stores:

- Backend API / monolith service
- Worker process(es) for BullMQ jobs
- PostgreSQL
- Redis
- Elasticsearch
- Stripe as external system
- Optional reverse proxy / API gateway / load balancer
- Optional admin UI

Why

C4 Container diagrams show the major technology building blocks, their responsibilities, and how they communicate. [\[c4model.com\]](https://c4model.com)

Important nuance

In a modular monolith, your “containers” are usually:

- **Monolith API process**
- **Background worker process**
- **Postgres**
- **Redis**
- **Elasticsearch**
- External services

Inside the monolith itself, the **module decomposition** belongs in the next-level component views.

Step 7) Create dynamic/runtime views for critical flows

Goal

Document how the system behaves at runtime for important business flows.

Deliverables

Sequence diagrams or dynamic views for:

- User signup/login
- Protected API request with JWT + authorization
- Entity creation/update
- Search indexing flow
- Background job flow
- Payment checkout flow
- Stripe webhook flow
- Failure/retry flow

Tools

- Mermaid sequence diagrams / PlantUML / Structurizr dynamic views

Why

C4 explicitly includes **Dynamic diagrams** as a supporting diagram type, and arc42 includes a **Runtime View** because static diagrams alone never explain real system behavior.

[\[c4model.com\]](https://c4model.com/), [\[arc42.org\]](https://arc42.org/)

What these runtime views should answer

For each flow:

- Who initiates it?
 - Which module handles it?
 - Where is the transaction boundary?
 - Which event is published?
 - Which queue/job is created?
 - Which read model gets updated?
 - What happens if a step fails?
 - Which operations are retried?
 - Which steps are idempotent?
-

Step 8) Design the event-driven architecture inside the modular monolith

Goal

Formalize your events so the monolith remains modular rather than turning into a big ball of mud.

Deliverables

- event-catalog.md
- domain-events.md
- integration-events.md
- Optional AsyncAPI document for async contracts
- Failure/retry policy spec

Tools

- Markdown
- AsyncAPI (optional but useful)
- Mermaid sequence diagrams
- ADRs for event strategy
- Queue/job definitions in code

What to define

For each event:

- Event name
- Producer module
- Consumers
- Trigger
- Payload schema
- Delivery mechanism (in-memory domain event, DB outbox, BullMQ job, webhook ingestion)
- Idempotency strategy
- Ordering expectations
- Retry policy
- Dead-letter / poison-event handling
- Observability fields (correlation ID, causation ID, tenant ID, actor ID)

Recommended split

Document at least two categories:

1. **Domain events**

Example: order.created, payment.requested, user.role.changed

2. **Integration / async processing events**

Example: search.reindex.requested, stripe.webhook.received,
notification.send.email

Key design decision to document

If you use Postgres + event-driven architecture in a modular monolith, strongly consider an **outbox pattern** (or equivalent durable event publication mechanism) to avoid “DB write succeeded but event publish failed.”

Step 9) Design the relational data model (PostgreSQL)

Goal

Turn the domain model into a relational schema that supports correctness first, then performance.

Deliverables

- Conceptual data model
- Logical data model
- Physical schema
- ERD
- Naming conventions
- Migration plan

Tools

- **dbdiagram.io / DBML**
- **pgModeler**
- **DBeaver**
- **draw.io**
- TypeORM entity definitions (if that's your ORM)
- SQL migration files

Documentation outputs

- data-model.md
- schema-overview.md
- erd.dbml
- erd.png
- naming-conventions.md

- migrations/ in code

How to do it in order

9.1 Conceptual model

Define major entities and relationships without columns yet.

9.2 Logical model

Define:

- Tables
- PKs
- FKs
- Cardinality
- Unique constraints
- Optionality
- Status enums
- Ownership rules

9.3 Physical model

Define:

- Column types
- Indexes
- Partitioning (if needed)
- Audit columns
- Multi-tenancy columns
- Soft-delete strategy
- Check constraints
- Trigger strategy (if used)
- created_at, updated_at, versioning / optimistic locking fields

What the documentation should explicitly include

For each table:

- Purpose
- Owning module
- Columns and meaning
- Constraints
- Indexes
- Data lifecycle
- Sensitive fields / PII classification
- Who can read/write it

Practical database design guidance

For a production-grade backend doc, also include:

- Which writes must be transactional
 - Which relations are strict FK-enforced vs eventual/read-model-linked
 - Which aggregates must remain strongly consistent
 - High-cardinality query patterns
 - Index rationale
-

Step 10) Design Redis + BullMQ job architecture

Goal

Specify what goes to the queue system and what does not.

Deliverables

- queues-and-jobs.md
- Worker responsibilities
- Retry/backoff policy
- Queue naming conventions
- DLQ/failure handling doc

Tools

- BullMQ
- Redis
- Mermaid sequence diagrams
- Markdown

Why

BullMQ is designed as a Redis-based queue system for background processing, with retries, delayed jobs, concurrency, and horizontally scalable workers; its own docs note that in the worst case delivery is **at least once**, so your consumers should be written accordingly.

[\[docs.bullmq.io\]](https://docs.bullmq.io)

What to document

For each queue:

- Queue name
- What creates jobs
- Job payload schema
- Worker name

- Concurrency
- Retry count
- Backoff
- Timeout
- Idempotency key
- Dead-letter behavior
- Monitoring metrics

Typical queues in your type of system

- search-indexing
- notifications-email
- notifications-sms
- stripe-webhook-processing
- billing-reconciliation
- audit-log-fanout
- report-generation

Important rule

Only queue tasks that are:

- asynchronous,
 - retryable,
 - and safe/acceptable to run more than once if duplicates happen.
-

Step 11) Design the Elasticsearch search model separately from Postgres

Goal

Treat Elasticsearch as a **read/search model**, not as your source of truth.

Deliverables

- search-architecture.md
- Index list
- Mapping definitions
- Indexing strategy
- Reindex strategy

Tools

- Elasticsearch mappings/templates JSON

- Kibana / Elastic tools
- Markdown

Why

Elastic recommends **explicit mappings for production** because they provide full control over how data is indexed; mapping changes can be limited later and some changes require **reindexing**, while uncontrolled dynamic fields can cause **mapping explosion**. [\[elastic.co\]](https://www.elastic.co/guide/en/elasticsearch/reference/current/mapping-explosion.html)

What to document

For each index:

- Index name
- Purpose
- Source tables / source modules
- Mapping
- Text analyzers
- Keyword fields
- Nested/object structure
- Filtering/sorting fields
- Sync trigger (event/job)
- Refresh expectations
- Rebuild/reindex strategy
- Ownership and versioning

Design principle

Postgres schema and Elasticsearch index schema are **different models**:

- Postgres = normalized transactional model
- Elasticsearch = denormalized search-optimized model

Document the transform between them explicitly.

Step 12) Design authentication, authorization, and RBAC (plus policy layer)

Goal

Define exactly how identity and permissions work.

Deliverables

- authentication.md

- authorization.md
- rbac-model.md
- Token design doc
- Permission matrix

Tools

- Markdown
- Sequence diagrams
- Optional threat modeling tool
- Security review checklist

What to document for JWT

- Access token lifetime
- Refresh token strategy
- Signing algorithm
- Claims (sub, iss, aud, exp, nbf, jti, roles, tenant/context)
- Revocation / rotation strategy
- Token validation rules
- Service-side middleware/guard flow

Why

OWASP ASVS 5.0 explicitly includes dedicated chapters for **Authorization (V8)** and **Self-contained Tokens (V9)**, meaning both should be documented and verified separately.

[\[cheatsheet....owasp.org\]](#), [\[asvs.dev\]](#)

OWASP authorization guidance recommends:

- **least privilege**,
- **deny by default**,
- checking authorization on **every request**,
and notes that app teams should often prefer richer policy models (ABAC/ReBAC) over pure RBAC when the domain becomes more complex. [\[cheatsheet....owasp.org\]](#)

OWASP ASVS guidance for self-contained tokens requires:

- validating signature/MAC,
- allowlisting algorithms,
- avoiding the None algorithm,
- validating the token validity window and intended usage/context. [\[asvs.dev\]](#), [\[cheatsheet....owasp.org\]](#)

Practical recommendation

Use:

- **JWT for authentication/session representation**
- **RBAC as the coarse-grained base**
- **resource-level policy checks** for real business authorization

Example:

- JWT says user is admin
- policy layer checks whether this admin belongs to the same tenant/org, owns the resource, and is allowed to perform this action

That prevents “RBAC-only” designs from becoming too coarse.

Step 13) Design Stripe payment flow and webhook processing

Goal

Document payment initiation, confirmation, reconciliation, and failure handling end to end.

Deliverables

- payments-architecture.md
- stripe-integration.md
- webhook-events.md
- Payment state machine
- Reconciliation strategy
- Idempotency approach

Tools

- Stripe docs
- Sequence diagrams
- Markdown
- Queue workers

What to document

Payment initiation

- Which API creates checkout/payment intent/session
- What metadata is attached
- Which internal entity the payment references
- Temporary payment states

Webhook ingestion

- Endpoint
- Signature verification
- Event storage
- Deduplication
- Ack strategy
- Queueing/inbox strategy

Reconciliation

- Which Stripe events matter
- Which internal state transitions happen
- What is source of truth for “paid”
- What happens if internal DB update fails after receipt
- Recovery job

Critical operational rules

Stripe says webhook handlers should:

- accept POST requests,
 - use HTTPS in production,
 - verify signatures using the **raw request body**,
 - return a **2xx quickly before complex logic**,
 - and expect automatic retries for failed deliveries for **up to three days** in live mode.
- [\[docs.stripe.com\]](https://docs.stripe.com)

Recommended production design

1. Receive webhook
2. Verify signature
3. Persist event receipt + dedupe key
4. Return 200 quickly
5. Process event asynchronously via BullMQ
6. Apply idempotent domain updates
7. Mark event processed

This design matches both Stripe’s operational expectations and BullMQ’s async background-processing strengths. [\[docs.stripe.com\]](https://docs.stripe.com), [\[docs.bullmq.io\]](https://docs.bullmq.io)

Step 14) Create deployment and infrastructure views

Goal

Show where everything runs and how it is connected.

Deliverables

- deployment-view.mmd
- infrastructure-topology.md

Tools

- C4 Deployment diagram tooling
- diagrams.net / Structurizr / Mermaid
- Terraform diagrams if applicable

What to include

- API instances
- Worker instances
- PostgreSQL
- Redis
- Elasticsearch
- Load balancer / ingress
- Secrets/config mechanism
- Network boundaries
- Observability stack
- Environments (dev/stage/prod)

Why

C4 includes **Deployment diagrams** as a supporting diagram type, and arc42 includes a **Deployment View** because architecture is incomplete without showing runtime placement/infrastructure mapping. [\[c4model.com\]](https://c4model.com/), [\[arc42.org\]](https://arc42.org/)

Step 15) Add operational architecture: observability, resilience, and failure modes

Goal

Explain how the system behaves in production when things go wrong.

Deliverables

- operability.md

- failure-modes.md
- monitoring-and-alerting.md
- runbooks.md

Tools

- Markdown
- Metrics/logging/tracing stack docs
- Incident runbooks

What to document

- Correlation IDs across sync + async flows
- Metrics per module/queue/webhook
- Alert thresholds
- Retry policies
- Circuit breaker / timeout strategy
- Dead-letter handling
- Reindex/replay procedures
- Stripe webhook replay strategy
- Disaster recovery notes

Practical examples

Document:

- What if Redis is unavailable?
- What if Stripe sends duplicates?
- What if Elasticsearch indexing lags?
- What if the worker crashes after DB write but before marking queue job complete?
- What if access token is valid but underlying user permissions changed?

This is what separates “drawings” from real architecture.

Step 16) Review the design with architecture checklists and threat modeling

Goal

Validate that the architecture is not just complete, but safe and implementable.

Deliverables

- Architecture review checklist
- Threat model

- Open risk register
- Updated ADRs

Tools

- Team review session
- OWASP ASVS checklist
- OWASP authorization checklist
- Threat modeling tool / diagram review session

What to review

- Security
- Data consistency
- Scalability
- Operability
- Cost
- Extensibility
- Failure recovery
- Tenant isolation
- Data access boundaries

Why

OWASP ASVS is specifically intended to give teams a concrete set of security requirements for designing, developing, and testing modern web applications and services. [\[owasp.org\]](https://owasp.org), [\[github.com\]](https://github.com)

Step 17) Keep the docs living and tied to implementation

Goal

Prevent architecture docs from becoming outdated.

Deliverables

- “Definition of Done” update
- PR checklist
- Documentation ownership
- Change management policy

Tools

- Git PR template
- CI checks (optional)

- ADR process
- Wiki publish automation (optional)

Process rule

Any change to:

- a module boundary,
- a table schema,
- an event payload,
- an auth rule,
- a webhook flow,
- an index mapping,
- or a deployment topology

should require:

1. code change,
2. migration/contract change,
3. doc change,
4. ADR if architecturally significant.

That is the mature way to keep implementation and architecture aligned.

What the final documentation set should contain

If you do this properly, your final package should look something like this:

```
1 docs/  
2   architecture/  
3     00-overview.md  
4     01-requirements-and-goals.md  
5     02-constraints-and-assumptions.md  
6     03-stakeholders-and-context.md  
7     04-solution-strategy.md  
8  
9     context/  
10      system-context.mmd  
11      system-context.png  
12  
13     containers/  
14      container-diagram.mmd
```

```
15         container-responsibilities.md
16
17     modules/
18         module-map.md
19         module-dependencies.md
20         component-diagrams/
21
22     runtime/
23         auth-login-sequence.mmd
24         create-order-sequence.mmd
25         payment-checkout-sequence.mmd
26         stripe-webhook-sequence.mmd
27         search-indexing-sequence.mmd
28
29     data/
30         conceptual-data-model.md
31         logical-data-model.md
32         physical-schema.md
33         erd.dbml
34         erd.png
35         naming-conventions.md
36         indexing-strategy.md
37
38     events/
39         event-catalog.md
40         async-contracts.md
41         retry-and-idempotency.md
42
43     search/
44         elastic-indexes.md
45         mappings/
46         reindex-strategy.md
47
48     security/
49         authentication.md
50         authorization.md
51         jwt-design.md
52         permission-matrix.md
53         threat-model.md
54
```

```
55     payments/
56         stripe-integration.md
57         webhook-handling.md
58         payment-state-machine.md
59         reconciliation.md
60
61     deployment/
62         deployment-view.mmd
63         topology.md
64         environments.md
65
66     operations/
67         observability.md
68         monitoring-and-alerting.md
69         runbooks.md
70         resilience-and-failure-modes.md
71
72     adr/
73         0001-use-modular-monolith.md
74         0002-use-postgres.md
75         0003-use-bullmq.md
76         0004-use-elasticsearch.md
77         0005-use-event-driven-internal-architecture.md
78         0006-use-jwt.md
79         0007-use-rbac-plus-policies.md
80         0008-use-stripe-webhooks.md
81
82     api/
83         openapi.yaml
84
85     db/
86         migrations/
87         seeds/
```

Where to store the documentation

Best practice (recommended)

Store architecture docs in the same Git repo as the backend codebase as docs-as-code.

Why

- version controlled
- reviewable in PRs
- easy to diff
- tied to schema changes
- tied to event/API changes
- easy to keep current

This is especially consistent with ADR guidance, which commonly recommends storing decision records in the repository itself. [\[github.com\]](https://github.com), [\[adr.github.io\]](https://adr.github.io)

Optional second layer

Publish a rendered/readable version to:

- Confluence
- Notion
- Azure DevOps Wiki
- GitHub Pages

Rule

Use the **Git repo as the source of truth**.

Use the wiki/portal only as a distribution/readability layer.

Recommended tool stack for your exact scenario

For a backend software engineer doing this on a real project, I'd recommend:

Written docs

- **Markdown** in repo
- Optional: **AsciiDoc with arc42 template** if the team likes more formal docs [\[arc42.org\]](https://arc42.org)

Diagrams

- **Structurizr DSL** if you want serious architecture-as-code
- Otherwise **Mermaid** for lightweight PR-friendly diagrams
- **draw.io** for quick collaborative visuals

ADRs

- Markdown files in docs/architecture/adr/

- Nygard or MADR-style template

Database schema

- **DBML/dbdiagram.io** or **pgModeler**
- SQL migrations in repo
- ERD exports committed to docs

API contracts

- **OpenAPI/Swagger**

Event contracts

- Markdown event catalog
- Optional **AsyncAPI**

Security reviews

- OWASP ASVS checklist
 - Threat model diagram & abuse-case review [\[owasp.org\]](https://owasp.org), [\[cheatsheet....owasp.org\]](https://cheatsheet....owasp.org)
-

If you want a simple “real-world execution order”

Here’s the shortest practical order I’d actually follow on a project:

1. Read brief and extract requirements/NFRs
 2. Write assumptions + open questions
 3. Create initial ADRs
 4. Draw C4 System Context
 5. Define modules/bounded contexts for the modular monolith
 6. Draw Container diagram
 7. Design the main runtime flows
 8. Design Postgres schema + ERD
 9. Define events + queues + worker model
 10. Define Elasticsearch read models + mappings
 11. Define JWT/authz/RBAC/policy layer
 12. Define Stripe payment + webhook handling
 13. Define deployment + observability + failure strategy
 14. Run review / threat model / update ADRs
 15. Keep everything in Git and require doc updates in PRs
-

If you want, I can do the **next practical step** for you and generate either:

1. a **ready-to-use architecture documentation template** (folder structure + markdown skeletons), or
2. a **full sample system architecture document** for your backend stack (modular monolith + Postgres + Redis/BullMQ + Elasticsearch + JWT + RBAC + Stripe).